Git & GitHub – Erste Schritte

Anleitung für unser Testrepo **ingenieur-toolbox**

Worum geht's? Wir arbeiten im Softwareprojekt gemeinsam an Code. Damit das nicht über USB-Sticks und Mails mit „final_v3_wirklich_final.py“ läuft, nutzen wir **Git** (Versionsverwaltung auf deinem Rechner) und **GitHub** (die Online-Plattform, auf der das gemeinsame Repo liegt). Mit dem Testrepo übst du den kompletten Ablauf einmal gefahrlos – kaputt machen kannst du nichts.

Ablauf im Überblick

1	Einrichten	VS Code, Python, Git installieren, GitHub-Account anlegen
2	Klonen	Repo von GitHub auf den eigenen Rechner holen
3	Branch	Eigenen Arbeitszweig anlegen
4	Erweitern	Ein neues Werkzeug programmieren (selbst oder mit KI)
5	Commit & Push	Änderung speichern und hochladen
6	Pull Request	Änderung zur Übernahme in main vorschlagen

Zeitaufwand: ca. 30–45 Minuten beim ersten Mal.

Kleines Wörterbuch

Repository (Repo)	Projektordner, dessen komplette Änderungsgeschichte Git speichert
Clone	Einmaliges Herunterladen eines Repos von GitHub
Commit	Ein gespeicherter Zwischenstand mit kurzer Beschreibung
Push / Pull	Commits zu GitHub hochladen / neue Commits von GitHub holen
Branch	Eigener Arbeitszweig – du arbeitest, ohne main zu stören
main	Der Hauptzweig. Hier liegt nur geprüfter, funktionierender Code
Pull Request (PR)	Antrag auf GitHub: „Bitte meinen Branch in main übernehmen“

1. Einrichten (einmalig)

1.1 Visual Studio Code installieren

- Download: **https://code.visualstudio.com** – Version für dein Betriebssystem wählen und installieren.
- Windows: Bei der Installation die Häkchen *„Zu PATH hinzufügen“* und *„Mit Code öffnen“* setzen.
- VS Code starten, links auf das Erweiterungen-Symbol (vier Quadrate) klicken, nach **Python** (von Microsoft) suchen und installieren.
- Optional: Sprachpaket **German Language Pack** – die Anleitung nennt die Menüs auf Deutsch und Englisch.

1.2 Python installieren

- Download: **https://www.python.org/downloads** (aktuelle Version 3.x).
- Windows: Im ersten Installationsfenster unbedingt **„Add python.exe to PATH“** anhaken!
- macOS/Linux: Python ist meist schon da. Prüfen mit `python3 --version`.

1.3 Git installieren

- Windows: **https://git-scm.com/downloads** – Installer ausführen, alle Standardeinstellungen übernehmen (einfach „Next“ klicken).
- macOS: Im Terminal `git --version` eingeben – macOS bietet dann die Installation an.
- Linux: `sudo apt install git` (Ubuntu/Debian).

Danach VS Code **neu starten** und ein Terminal öffnen: Menü **Terminal > Neues Terminal** (*Terminal* > *New Terminal*). Prüfen, ob alles da ist:

```
git --version
python --version        (Windows evtl.: py --version, macOS/Linux: python3 --version)
```

1.4 Git sagen, wer du bist

Jeder Commit wird mit Name und E-Mail gespeichert. Einmalig im Terminal eingeben – **dieselbe E-Mail wie bei GitHub** verwenden:

```
git config --global user.name "Max Mustermann"
git config --global user.email "max.mustermann@beispiel.de"
```

1.5 GitHub-Account anlegen

- Auf **https://github.com** > **Sign up**. E-Mail, Passwort und einen Benutzernamen wählen (am besten erkennbar, z. B. *max-mustermann*).
- E-Mail-Adresse bestätigen. Empfohlen: Zwei-Faktor-Authentifizierung aktivieren (Settings > Password and authentication).
- **Deinen GitHub-Benutzernamen an den Projektleiter schicken.** Du bekommst dann eine Einladung per Mail für das Repo – diese **annehmen**, sonst darfst du später nicht pushen.

2. Repo klonen (herunterladen)

Die Repo-Adresse findest du auf der GitHub-Seite des Repos über den grünen Button **<> Code** (HTTPS-Link kopieren). Sie sieht so aus:

```
https://github.com/<BENUTZERNAME>/ingenieur-toolbox.git
```

Variante A – mit VS Code (einfach):

- VS Code öffnen, links das Symbol **Quellcodeverwaltung** (*Source Control*, drei Punkte mit Linien) anklicken > **Repository klonen** (*Clone Repository*).
- Link einfügen, Enter, einen Speicherort wählen (z. B. *Dokumente/Projekte*).
- Beim ersten Mal öffnet sich der Browser zum Anmelden bei GitHub – erlauben.
- Auf **Öffnen** klicken. Fertig – du siehst links die Dateien.

Variante B – mit dem Terminal:

```
cd Dokumente
git clone https://github.com/<BENUTZERNAME>/ingenieur-toolbox.git
cd ingenieur-toolbox
code .                  (öffnet den Ordner in VS Code)
```

Programm testen – im VS-Code-Terminal (es muss im Ordner *ingenieur-toolbox* stehen):

```
python main.py          (Windows evtl.: py main.py / macOS, Linux: python3 main.py)
```

Es erscheint ein Menü mit Werkzeugen (Ohmsches Gesetz, Kreisfläche, …). Mit 0 beenden.

3. Eigenen Branch anlegen

Du arbeitest **nie direkt auf main**, sondern auf einem eigenen Zweig. Name-Schema: `feature/<vorname>-<thema>`

```
git switch main         (sicherstellen, dass du auf main bist)
git pull                (neuesten Stand holen)
git switch -c feature/max-biegemoment
```

In VS Code: unten links in der blauen Statusleiste steht der aktuelle Branch-Name. Ein Klick darauf > **Neuen Branch erstellen** (*Create new branch*) geht auch.

4. Code erweitern

Das Programm ist so gebaut, dass **jedes Werkzeug eine eigene Datei** im Ordner *werkzeuge/* ist. Neue Dateien erscheinen automatisch im Menü. Dadurch kommt ihr euch nicht in die Quere.

- `werkzeuge/_vorlage.py` kopieren (Rechtsklick > Kopieren, Einfügen) und umbenennen, z. B. in `biegemoment.py` – **ohne** Unterstrich am Anfang, keine Umlaute/Leerzeichen.
- `NAME`, `BESCHREIBUNG` und die Funktion `ausfuehren()` anpassen.
- Mit `python main.py` testen, bis es funktioniert.
- Ideen: Biegemoment, Leistung P = U·I, Drehmoment aus Leistung und Drehzahl, Temperatur- oder Druck-Umrechnung, Reihen-/Parallelschaltung … (Liste steht auch in der README.md). Sprecht kurz ab, wer was macht.

Mit KI (ChatGPT, Copilot, Claude …): Gib der KI die Vorlage und eine klare Aufgabe, z. B.:

```
Hier ist eine Vorlage für ein Python-Werkzeug (Inhalt von _vorlage.py einfügen).
Schreibe nach diesem Muster ein Werkzeug "Biegemoment", das für einen
Einfeldträger mit mittiger Einzellast F und Länge L das maximale
Biegemoment M = F*L/4 berechnet. Nutze zahl_eingeben() für die Eingaben.
```

Wichtig bei KI-Code: Du committest ihn unter deinem Namen – also verstehen, testen und Ergebnisse einmal von Hand nachrechnen. Niemals Passwörter, Kundendaten oder internen Projektcode in öffentliche KI-Tools kopieren.

5. Commit & Push

Variante A – Terminal:

```
git status                        (zeigt, was sich geändert hat)
git add werkzeuge/biegemoment.py (Datei für den Commit vormerken)
git commit -m "Werkzeug Biegemoment hinzugefügt"
git push -u origin feature/max-biegemoment
```

Beim nächsten Push auf demselben Branch reicht einfach `git push`.

Variante B – VS Code:

- Links **Quellcodeverwaltung** öffnen. Geänderte Dateien stehen unter *Änderungen*.
- Beim Überfahren der Datei auf das **+** klicken (= `git add`).
- Oben eine Commit-Nachricht eintippen und auf **Commit** klicken.
- Danach **Branch veröffentlichen** (*Publish Branch*) bzw. später **Änderungen synchronisieren** (*Sync Changes*) klicken = Push.

Gute Commit-Nachricht: kurz, im Präsens oder als Ergebnis, sagt *was* passiert ist. Gut: *„Temperatur-Umrechnung °C/K/°F hinzugefügt“*. Schlecht: *„update“, „asdf“, „fix“*.

6. Pull Request stellen

- Nach dem Push zeigt GitHub auf der Repo-Seite einen gelben Hinweis mit dem Button **Compare & pull request** – anklicken. (Alternativ: Reiter *Pull requests* > *New pull request*.)
- Base: **main**, Compare: **dein Branch**. Titel und 1–2 Sätze Beschreibung, was du gemacht hast.
- **Create pull request**. Ein Teamkollege schaut drüber (Review) und klickt auf **Merge**.
- Danach lokal wieder auf main wechseln und den neuen Stand holen:

```
git switch main
git pull
```

Herzlichen Glückwunsch – das ist der Ablauf, den wir im echten Projekt jeden Tag nutzen. Für die nächste Änderung wieder bei Schritt 3 starten.

Wenn es mal hakt

- **„rejected … fetch first“ beim Push:** Jemand war schneller. `git pull` ausführen, dann erneut pushen.
- **Merge-Konflikt:** Zwei Leute haben dieselbe Zeile geändert. VS Code markiert die Stelle; auswählen, welche Version bleibt (*Accept Current/Incoming*), speichern, committen. Im Zweifel: fragen!
- **„permission denied“ / 403:** Einladung zum Repo noch nicht angenommen (Mail prüfen).
- **„python“ wird nicht gefunden:** `py` bzw. `python3` probieren, oder Python mit Häkchen „Add to PATH“ neu installieren.

7. Das bitte NICHT machen

Nicht	Warum
X Direkt auf main pushen	main muss immer laufen. Immer Branch + Pull Request.
X git push --force	Überschreibt die Arbeit anderer unwiderruflich. Nur nach Absprache.
X Passwörter, API-Keys, .env committen	Bleiben in der Historie für immer sichtbar – auch nach dem Löschen.
X Große/generierte Dateien committen	Keine venv-Ordner, Build-Ausgaben, Videos, ZIPs. Dafür gibt's die .gitignore.
X Kaputten / ungetesteten Code pushen	Vor jedem Commit einmal laufen lassen.
X Riesen-Commits nach 2 Wochen	Lieber klein und oft committen – ein Thema pro Commit.
X Nichtssagende Commit-Messages	„update“ hilft in drei Monaten niemandem.
X Tagelang ohne git pull arbeiten	Vor Arbeitsbeginn pullen – das spart Konflikte.
X Dateikopien wie code_alt.py	Git ist das Backup. Alte Stände holt man über die Historie zurück.
X KI-Code blind übernehmen	Erst verstehen und testen, dann committen.

Spickzettel

```
git status                        Was hat sich geändert?
git pull                          Neuesten Stand holen
git switch -c feature/name-thema  Neuen Branch anlegen und wechseln
git switch main                   Zurück auf main
git add <datei>                   Datei für Commit vormerken
git commit -m "Nachricht"         Zwischenstand speichern
git push                          Hochladen
git log --oneline                 Historie anzeigen
```